

Technik Programista

Zaawansowane aplikacje webowe

MVC i MTV

Jak dzielić aplikację na części

Model – View – Controller, Model – Template – View, gdzie to jest we Flasku, Django, React

Klasa 4 | rok szkolny 2026/2027 | System: Windows

Spis treści

Spis treści

Spis treści.....	2
1. Problem: wszystko w jednym pliku	3
2. MVC – trzy warstwy.....	3
2.1 Analogia: restauracja	4
2.2 Zasady	4
3. Jak przechodzi żądanie przez MVC	4
4. MTV – ten sam pomysł w Django.....	5
5. Gdzie to jest w frameworkach z egzaminu	5
5.1 Flask nie narzuca MVC.....	6
6. Po co nam to naprawdę	6
7. Zadania	7
Zadanie 1: Podział na warstwy (na kartce)	7
Zadanie 2: Gdzie jest błąd?	7
Zadanie 3: Przepływ żądania.....	7
Zadanie 4: Słownik	7
Zadanie 5: Do repo	7

1. Problem: wszystko w jednym pliku

Wyobraź sobie sklep internetowy napisany tak: jeden plik, w którym w jednej funkcji jest zapytanie do bazy, obliczanie ceny z rabatem, sklejanie HTML-a ze stringów i wysyłanie maila. Działa. Do momentu, aż:

- grafik chce zmienić wygląd strony – musi grzebać w kodzie, który liczy rabaty,
- księgowość zmienia zasady rabatów – programista musi szukać tego w HTML-u,
- chcesz dodać aplikację mobilną – a cała logika jest wklejona w stronę WWW,
- chcesz napisać test – ale nie da się przetestować obliczania ceny bez uruchomienia całej strony.

Brzmi znajomo? **Dokładnie to miałeś na egzaminie INF.03.** Jeden plik PHP: na górze `<html>`, w środku `<?php` z połączeniem do bazy, zapytanie SQL sklezione ze stringów, pętla `while` wypływająca `<tr>` z wynikami, na dole `</html>`. Do egzaminu wystarczyło. Ale spróbuj w takim pliku zmienić wygląd tabeli, nie psując zapytania – albo dać go koledze, żeby dopisał drugą stronę.

```
<html><body>
<?php
    $db = mysqli_connect("localhost", "root", "", "sklep");
    $sql = "SELECT * FROM produkty WHERE cena < 100";
    $wynik = mysqli_query($db, $sql);
    while ($w = mysqli_fetch_array($wynik)) {
        echo "<tr><td>" . $w['nazwa'] . "</td>";
        echo "<td>" . $w['cena'] . "</td></tr>";
    }
?>
</body></html>
```

HTML, PHP i SQL w jednym pliku, w jednej funkcji. Model, widok i kontroler zlepione w jedno. Na INF.03 to była norma. W INF.04 – i w pracy – tak się nie pisze.

To się nazywa **kod spaghetti**. Każda zmiana ciągnie za sobą inne. Rozwiązanie jest stare (lata 70., język Smalltalk) i nazywa się **rozdzielenie odpowiedzialności** (separation of concerns): każda część programu robi jedną rzecz.

2. MVC – trzy warstwy

MVC to skrót od Model – View – Controller. Aplikację dzielimy na trzy części:

Warstwa	Za co odpowiada	Pytanie, na które odpowiada
Model	Dane i reguły biznesowe. Co to jest produkt, jaką ma cenę, jak liczyć rabat, jak zapisać do bazy.	Co aplikacja wie i umie?
View (widok)	Prezentacja. HTML, szablon, wygląd. Pokazuje dane, nie liczy ich.	Jak to wygląda?
Controller (kontroler)	Pośrednik. Odbiera żądanie, pyta model o dane, przekazuje je do widoku, zwraca odpowiedź.	Kto obsługuje to, co przyszło?

2.1 Analogia: restauracja

- **Model** = kuchnia. Wie, jak zrobić danie, ma składniki (bazę danych), ma przepisy (logikę).
- **View** = talerz i menu. To, co klient widzi. Talerz nie gotuje.
- **Controller** = kelner. Przyjmuje zamówienie (żądanie), niesie do kuchni, przynosi danie na talerzu. Kelner nie gotuje i nie jest talerzem.

💡 *Test: jeśli kelner zaczyna gotować, to znaczy, że masz logikę w kontrolerze. Jeśli talerz sam liczy cenę – masz logikę w widoku. Oba to błędy.*

2.2 Zasady

- Model **nie wie**, że istnieje strona WWW. Ten sam model może obsłużyć stronę, aplikację mobilną i API.
- View **nie liczy**. Dostaje gotowe dane i je wyświetla. Może mieć pętlę i warunek, ale nie zapytanie do bazy.
- Controller jest **cienki**. Kilka linii: weź dane, oddaj widokowi. Jeśli kontroler ma 200 linii, coś jest w złym miejscu.

3. Jak przechodzi żądanie przez MVC

```

PRZEGŁĄDARKA
| 1. GET /produkt/5
v
CONTROLLER
| 2. daj mi produkt nr 5
v
MODEL  -- 3. SELECT -->  BAZA DANYCH
| 4. obiekt Produkt(nazwa='Laptop')
v
CONTROLLER
| 5. wyświetl ten produkt
v
VIEW   (szablon HTML + dane = gotowa strona)
| 6. HTML
v
PRZEGŁĄDARKA

```

1. Użytkownik klika link. Przeglądarka wysyła żądanie HTTP: **GET** /produkt/5.
2. Kontroler odbiera żądanie. Widzi, że chodzi o produkt nr 5.
3. Kontroler prosi model: „daj mi produkt o id 5”. Model pyta bazę danych.
4. Model zwraca obiekt **Produkt**. Kontroler nic o SQL nie wie.
5. Kontroler przekazuje obiekt do widoku: „wyświetl to”.
6. Widok wkłada dane w szablon HTML i oddaje gotową stronę. Przeglądarka ją pokazuje.

Zwróć uwagę: **przeglądarka rozmawia tylko z kontrolerem**. Nigdy bezpośrednio z modelem ani bazą.

4. MTV – ten sam pomysł w Django

Django (framework, którego użyjemy po Flasku) używa nazwy **MTV**: Model – Template – View. To **nie jest inny wzorzec**, to te same trzy warstwy pod innymi nazwami:

MVC	MTV (Django)	Co to jest w praktyce
Model	Model	klasa w <code>models.py</code> , tabela w bazie
View	Template	plik <code>.html</code> z szablonem
Controller	View	funkcja w <code>views.py</code> , która obsługuje URL

⚠ **Pułapka: „View” w Django to NIE to samo co „View” w MVC. View w Django = Controller w MVC. Na egzaminie i w rozmowie z kimś z ASP.NET musisz wiedzieć, o którym „View” mowa.**

Dlaczego Django tak to nazwało? Twórcy uznali, że w aplikacji webowej „widok” to funkcja decydująca, **które dane** zobaczysz, a „szablon” to tylko **jak** je zobaczysz. Trochę racji mają, ale zamieszanie zostało na lata.

5. Gdzie to jest w frameworkach z egzaminu

Podstawa programowa INF.04 wymienia: ASP.NET Core, Django, Angular, React.js, Node.js. Każdy z nich ma MVC, ale inaczej:

Framework	Model	View (prezentacja)	Controller (obsługa żądań)
Flask	sam piszesz (klasy, SQLAlchemy)	szablony Jinja2 w <code>templates/</code>	funkcje z <code>@app.route</code>
Django	<code>models.py</code>	<code>templates/*.html</code>	<code>views.py</code> + <code>urls.py</code>
ASP.NET Core MVC	<code>Models/</code>	<code>Views/*.cshtml</code>	<code>Controllers/*Controller.cs</code>
Node.js (Express)	sam piszesz (np. Mongoose)	szablony (EJS) albo brak	<code>app.get('/...', ...)</code>
React / Angular	stan komponentu, dane z API	komponenty (JSX / HTML)	obsługa zdarzeń w komponencie

React i Angular to frameworki **frontendowe** – działają w przeglądarce. Nie mają własnej bazy danych. Ich „model” to dane pobrane z API, które wystawia backend (Flask, Django, ASP.NET). Dlatego w tym roku najpierw robimy backend, potem frontend, a na końcu je łączymy.

5.1 Flask nie narzuca MVC

Flask to **mikroframework**. Daje routing i szablony, resztę organizujesz sam. Można we Flasku napisać kod spaghetti w jednym pliku – i przez pierwsze dwa materiały tak właśnie zrobimy, żeby zobaczyć, jak to działa od środka. Potem sami rozdzielimy to na warstwy. Django wymusza MTV od pierwszej minuty – i wtedy zrozumiesz, dlaczego.

6. Po co nam to naprawdę

- **Testowanie** – model bez WWW testuje się jedną funkcją. Sklejone razem: trzeba klikać w przeglądarce.
- **Praca zespołowa** – frontend robi widoki, backend modele, nikt nikomu nie psuje plików.
- **Wiele klientów** – jeden model, wiele widoków: strona, apka mobilna, API dla partnera.
- **Zmiana technologii** – wymieniasz szablony na React, model zostaje.
- **Egzamin i rozmowa o pracę** – „co to jest MVC” to pytanie z pierwszej piątki. Odpowiedź „model to dane, widok to HTML, kontroler łączy” wystarczy. Umieć to pokazać w swoim kodzie – to jest różnica.

7. Zadania

Zadanie 1: Podział na warstwy (na kartce)

Dla swojej domeny projektu (sklep / portal / rezerwacje / ogłoszenia) wypisz po 3 elementy, które należą do Modelu, do Widoku i do Kontrolera. Przykład dla sklepu: Model – **Produkt**, **Koszyk**, obliczanie rabatu; Widok – strona listy produktów, strona koszyka, e-mail z potwierdzeniem; Kontroler – obsługa **/produkty**, **/koszyk/dodaj**, **/zamow**.

Zadanie 2: Gdzie jest błąd?

Przeczytaj opisy i powiedz, która warstwa robi cudzą robotę:

1. Szablon HTML wykonuje zapytanie SQL, żeby pobrać listę użytkowników.
2. Funkcja obsługująca **/zamow** ma 150 linii: sprawdza stan magazynu, liczy VAT, generuje PDF i wysyła maila.
3. Klasa **Produkt** ma metodę **wyswietl_jako_html()**, która zwraca `<div>...</div>`.

Zadanie 3: Przepływ żądania

Narysuj (jak w rozdziale 3) przepływ żądania **POST /koszyk/dodaj** z produktem nr 7. Zaznacz, co robi kontroler, o co pyta model, co dostaje widok.

Zadanie 4: Słownik

Uzupełnij tabelę: dla Django, ASP.NET Core i Flaska podaj nazwę pliku lub folderu, w którym leży Model, Widok (prezentacja) i Kontroler. Bez zaglądania do rozdziału 5.

Zadanie 5: Do repo

Odpowiedzi z zadań 1–4 zapisz w pliku **mvc.md** w repozytorium swojego projektu. Commit: **Notatki MVC/MTV**. Push.